HANDS-ON LAB · OPENSTACK COMPUTE & IMAGE

LAB03

Compute & Images: Nova and Glance

Module « Infrastructures cloud natives » · 2026-2027

DOCUMENT CONTENTS

00 Objectives
P0 Part 0 — Checking the OpenStack environment
P1 Part 1 — Glance: managing images
P2 Part 2 — Flavors
P3 Part 3 — Boot-time customization with cloud-init
P4 Part 4 — Instance lifecycle
P5 Part 5 — Instance snapshots
P6 Part 6 — Hypervisor side: what Nova created
→ Cleanup

MODULE
Infrastructures cloud natives

PLATFORM
OpenStack · Nova · Glance

YEAR
2026-2027

00 Objectives

- Manage images with **Glance**: import, formats, properties, visibility and sharing between projects;
- Define **flavors** tailored to a given need;
- Customize an instance at boot time with **cloud-init**;
- Drive the **lifecycle** of a Nova instance (stop, pause, suspend, shelve, resize) and explain the effect of each operation;
- Create an **instance snapshot** and redeploy from it;
- Observe on the hypervisor side (**libvirt / KVM**) what Nova actually created.

Part 0 Checking the OpenStack environment

If the EC2 instance was stopped since the last session, check that the OpenStack services are running again.

TERMINAL

```
sudo systemctl list-units 'devstack@*' --no-pager
source ~/devstack/openrc admin admin      # adjust the path if needed
openstack compute service list
openstack hypervisor list
openstack network agent list
```

All services must be `up` / `enabled`. If a service is stopped:

TERMINAL

```
sudo systemctl restart devstack@<service>
```

Then check the external bridge `br-ex`, which connects the host to the `public` network (172.24.4.0/24) used by floating IPs:

TERMINAL

```
ip addr show br-ex
```

The bridge must be `UP` and carry the address `172.24.4.1/24`. DevStack configures this address during installation, but it is **not kept after a reboot** of the EC2 instance. If it is missing, restore it:

TERMINAL

```
sudo ip addr add 172.24.4.1/24 dev br-ex
sudo ip link set br-ex up
```

> **Q1**
> Which Nova services are listed by `openstack compute service list`? Explain the role of each one in one sentence (`nova-scheduler`, `nova-conductor`, `nova-compute`).

Part 1 Glance: managing images

1.1 Exploring the existing images

TERMINAL

```
openstack image list
openstack image show <cirros-image-id>
```

> **Q2**
> Write down the disk format (`disk_format`), the container format (`container_format`), the size, the visibility and the owner of the CirrOS image.

1.2 Importing an Ubuntu cloud image

TERMINAL

```
wget https://cloud-images.ubuntu.com/minimal/releases/noble/release/ubuntu-24.04-minimal-cloudimg-amd64.img
qemu-img info ubuntu-24.04-minimal-cloudimg-amd64.img
```

> **Q3**
> What is the actual format of the file according to `qemu-img info` ? Compare its size on disk (*disk size*) with its virtual size (*virtual size*). Why do these two values differ?

Create the image in Glance:

TERMINAL

```
openstack image create ubuntu-24.04-minimal \
    --file ubuntu-24.04-minimal-cloudimg-amd64.img \
    --disk-format qcow2 \
    --container-format bare \
    --min-disk 4 \
    --min-ram 512 \
    --property os_distro=ubuntu \
    --property os_version=24.04 \
    --private
```

Check where Glance stored the image (the `file` backend is the DevStack default):

TERMINAL

```
openstack image show ubuntu-24.04-minimal -c id -c status -c checksum
sudo ls -lh /opt/stack/data/glance/images/
```

> **Q4**
> What is the name of the file stored by Glance? How is it related to the image? What are `--min-disk` and `--min-ram` used for?

1.3 Visibility and sharing between projects

Glance offers four visibility levels: `private` , `shared` , `community` , `public` .

a) Create a test project and user

With the `admin` credentials:

TERMINAL

```
source ~/devstack/openrc admin admin
openstack project create project-lab03
openstack user create student-lab03 --project project-lab03 --password Lab03Pass
openstack role add --project project-lab03 --user student-lab03 member
```

> **Important:** an administrator sees all images, whatever their visibility. To actually observe the effect of sharing, you must look at things from the point of view of a **non-administrator** user.

b) Check that the image is invisible to the new project

Open a **second terminal** on the EC2 instance and log in as `student-lab03` :

TERMINAL

```
source ~/devstack/openrc admin admin
export OS_USERNAME=student-lab03
export OS_PASSWORD=Lab03Pass
export OS_PROJECT_NAME=project-lab03
openstack token issue -c project_id -c user_id
openstack image list
```

c) Share the image with the project

In the **first terminal** (admin):

TERMINAL

```
openstack image set --shared ubuntu-24.04-minimal
openstack image add project ubuntu-24.04-minimal project-lab03
openstack image member list ubuntu-24.04-minimal
```

d) Accept the share

In the **second terminal** (`student-lab03`):

TERMINAL

```
openstack image list
openstack image set --accept <ubuntu-image-id>
openstack image list
```

Check the member status again in the first terminal:

TERMINAL

```
openstack image member list ubuntu-24.04-minimal
```

> **Q5**
> Is the image visible to `student-lab03` **before** sharing? After sharing but **before** acceptance? After acceptance? What is the member status (`pending` , `accepted` , `rejected`) at each step?

> **Q6**
> What is the difference between a `shared` image and a `community` image? Who can make an image `public`, and why is this operation restricted to that role?

Part 2 Flavors

TERMINAL

```
openstack flavor list
```

Create two flavors (with the `admin` credentials):

TERMINAL

```
openstack flavor create lab03.small  --vcpus 1 --ram 1024 --disk 5
openstack flavor create lab03.medium --vcpus 2 --ram 2048 --disk 8
```

Create a private flavor, restricted to the Lab 03 project (`project-lab03`):

TERMINAL

```
openstack flavor create lab03.private --vcpus 1 --ram 512 --disk 4 --private
openstack flavor set lab03.private --project project-lab03
```

> **Q7**
> Can the Ubuntu image be launched with the `m1.tiny` flavor? Test it, then explain the message you get in relation to `--min-disk` and `--min-ram`.

> **Q8**
> Does the `lab03.private` flavor appear in `openstack flavor list` when you are logged in with another project? What is a private flavor used for in an enterprise cloud?

Part 3 Boot-time customization with cloud-init

All commands in this part are run on the EC2 instance, with the `admin` credentials (first terminal).

Go to your home directory: the key and the user-data file will be created there.

TERMINAL

```
cd ~
```

3.1 Preparing the network and access to the instance

a) Private network, subnet and router

TERMINAL

```
openstack network create net-lab03
openstack subnet create subnet-lab03 \
    --network net-lab03 \
    --subnet-range 192.168.30.0/24 \
    --dns-nameserver 8.8.8.8
openstack router create router-lab03
openstack router set router-lab03 --external-gateway public
openstack router add subnet router-lab03 subnet-lab03
```

b) Security group

Allow SSH, HTTP and ping:

TERMINAL

```
openstack security group create sg-lab03 --description "SSH, HTTP and ICMP for Lab 03"
openstack security group rule create --protocol tcp --dst-port 22 sg-lab03
openstack security group rule create --protocol tcp --dst-port 80 sg-lab03
openstack security group rule create --protocol icmp sg-lab03
openstack security group rule list sg-lab03
```

c) Key pair

TERMINAL

```
openstack keypair create cle-lab03 > cle-lab03.pem
chmod 600 cle-lab03.pem
```

3.2 Writing the user-data file

The file must be created in the same directory as the key (your home directory), because the `openstack server create` command in section 3.3 refers to it:

TERMINAL

```
cd ~
nano web-init.yaml
```

Paste the following content, then save and exit:

WEB-INIT.YAML (CLOUD-CONFIG)

```
#cloud-config
hostname: vm-web
write_files:
  - path: /var/www/index.html
    content: |
      <h1>Lab 03 - Instance configuree par cloud-init</h1>
runcmd:
  - cd /var/www && nohup python3 -m http.server 80 > /var/log/web.log 2>&1 &
```

> **Mind the indentation:** YAML is whitespace-sensitive. Use spaces, never tabs.

Meaning of the directives:

Directive	Role
`#cloud-config`	**Mandatory** first line: tells cloud-init that the file uses the cloud-config format (YAML)
`hostname`	Hostname assigned to the VM
`write_files`	Creates files in the VM (here, the website home page)
`runcmd`	Commands run as `root` at the end of the first boot (here, starting a Python web server on port 80)

Check the file:

TERMINAL

```
ls -l web-init.yaml
cat web-init.yaml
sudo cloud-init schema --config-file web-init.yaml
```

The last command validates the syntax of the file with the cloud-init tool installed on the EC2 instance. The expected result is `Valid schema web-init.yaml`.

> **Security:** the user-data content remains readable by any process in the VM for its entire lifetime. Never put a password, a private key or an access token in it.

3.3 Launching the instance

a) Create the instance

TERMINAL

```
openstack server create vm-web \
    --image ubuntu-24.04-minimal \
    --flavor lab03.small \
    --network net-lab03 \
    --security-group sg-lab03 \
    --key-name cle-lab03 \
    --user-data web-init.yaml \
    --wait
```

The `--wait` option returns once the instance is `ACTIVE`. At this point, it only has a **private** address (`192.168.30.x`).

b) Associate a floating IP

A floating IP is a Neutron resource, separate from the instance: it is first allocated from the `public` network, then associated with the VM. The address is stored in the `FIP` variable, reused throughout the rest of the lab:

TERMINAL

```
FIP=$(openstack floating ip create public -f value -c floating_ip_address)
openstack server add floating ip vm-web $FIP
echo $FIP
```

c) Follow the boot process

The first boot may take several minutes. Run the following command repeatedly until you see the line `Cloud-init v. ... finished`:

TERMINAL

```
openstack console log show vm-web | grep -i cloud-init
```

d) Test the web service

If not already done (Part 0), bring up the external bridge `br-ex`, which connects the host to the floating IP network:

TERMINAL

```
sudo ip addr add 172.24.4.1/24 dev br-ex
sudo ip link set br-ex up
```

Define the `web` and `relance_web` functions, which test and restart the web server:

TERMINAL

```
web() {
  curl -sS -m 5 http://$FIP || echo "=> no response from the web server"
}

relance_web() {
  ssh -n -i ~/cle-lab03.pem \
      -o StrictHostKeyChecking=no -o ConnectTimeout=5 \
      ubuntu@$FIP \
      "cd /var/www && sudo nohup python3 -m http.server 80 > /tmp/web.log 2>&1 < /dev/null &" \
    && echo "=> web server restarted"
}
```

Check connectivity and the service:

TERMINAL

```
ping -c 3 $FIP
web
```

The expected result is the page `<h1>Lab 03 - Instance configuree par cloud-init</h1>`.

Troubleshooting: if `web` does not respond, the `curl` error message tells you where to look:

Error	Meaning	Action
Immediate refusal (*Failed to connect ... after 0 ms*)	The VM responds, but no service is listening on port 80	Restart the server: `relance_web`, then `web`
Timeout (*timed out*) or no ping reply	Network problem, or VM not booted yet	Check `br-ex` (Part 0), wait for the boot to finish (console), then try again

Part 4 Instance lifecycle

For each operation below, write down the Nova **status**, the `vm_state`, the `power_state`, and the state seen by the hypervisor.

All commands are run on the EC2 instance, with the `admin` credentials, in the **same terminal as in Part 3** (the `FIP` variable and the `web` and `relance_web` functions are already defined there).

To simplify observations, define a small `etat` function in your terminal:

TERMINAL

```
etat() {
  openstack server show vm-web -f shell \
    -c status -c flavor -c OS-EXT-STS:vm_state -c OS-EXT-STS:power_state -c OS-EXT-STS:task_state
  sudo virsh list --all
}
etat
```

> **Important:** lifecycle commands are **asynchronous**. They return immediately, while the operation continues in the background (`task_state` other than `None`). So run **one operation at a time**, run `etat` again until `task_state` is back to `None` , write down the values, **then** move on to the next operation. Chaining two commands (for example `stop` then `start` on the same line) causes a `409 Conflict` error, because the instance is still in transition.
> **New terminal?** Functions and the `FIP` variable only exist in the terminal where they were defined. In a new terminal, define `web` , `relance_web` and `etat` again, and retrieve the floating address:
> `FIP=$(openstack floating ip list --port $(openstack port list --server vm-web -f value -c ID) -f value -c "Floating IP Address")`

After each **reverse** operation (`start` , `unpause` , `resume` , `unshelve` , `reboot`), test the web server with `web` . If it does not respond, note it for Q9, then restart it with `relance_web` before moving on to the next operation.

a) Stop and start

TERMINAL

```
openstack server stop vm-web
etat                                # wait for: SHUTOFF, task_state None
openstack server start vm-web
etat                                # wait for: ACTIVE
web
```

b) Pause

TERMINAL

```
openstack server pause vm-web
etat
openstack server unpause vm-web
etat
web
```

c) Suspend

TERMINAL

```
openstack server suspend vm-web
etat
openstack server resume vm-web
etat
web
```

d) Shelve

The `openstack server shelve` command puts an instance "on the shelf": the instance is stopped, then **removed from the compute node**, in order to free the resources (vCPUs, RAM, local disk) it was

using there. Yet it still exists for OpenStack: it remains visible in `openstack server list` and keeps its identity (UUID, name, IP addresses, key pair). The reverse command, `openstack server unshelve`, brings it back onto a compute node and restarts it.

This is the right operation for an instance that will not be used for a long time but that you want to get back as it was later. Watch carefully what changes in Nova, in libvirt **and** in Glance.

TERMINAL

```
openstack server shelve vm-web
etat                                  # wait for: SHELVED_OFFLOADED
openstack image list                  # a new image has appeared
openstack server unshelve vm-web
etat                                  # wait for: ACTIVE
openstack image list
web
```

e) Reboots

TERMINAL

```
openstack server reboot --soft vm-web
etat
openstack server reboot --hard vm-web
etat
web
```

> **Q9**
> Fill in the following table:

Operation	Nova status	libvirt state	Where is the memory state?	CPU/RAM resources freed on the host?	Web server running after the reverse operation?
stop					
pause					
suspend					
shelve					

> **Q10**
> After `shelve`, a new image appears in Glance. Why? What happens during `unshelve`?

> **Q11**
> What is the difference between `reboot --soft` and `reboot --hard`?

4.1 Resize

Resize the instance to the `lab03.medium` flavor, then **revert** the operation:

TERMINAL

```
openstack server resize --flavor lab03.medium vm-web
etat                                  # wait for: VERIFY_RESIZE, task_state None
openstack server resize revert vm-web
etat                                  # wait for: ACTIVE — which flavor?
```

Do it again, but this time **confirm**:

TERMINAL

```
openstack server resize --flavor lab03.medium vm-web
etat                                  # wait for: VERIFY_RESIZE, task_state None
openstack server resize confirm vm-web
etat                                  # ACTIVE with lab03.medium
```

Finally, try to resize to a flavor with a **smaller** disk:

TERMINAL

```
openstack server resize --flavor m1.tiny vm-web
openstack image show ubuntu-24.04-minimal -c min_disk
```

Q12

What is the status of the instance right after the resize, before confirmation? What do `openstack server resize revert` and `openstack server resize confirm` do?

Q13

What happens when resizing to `m1.tiny`, and why? Compare the minimum size given in the error message with the `min_disk` of the Ubuntu image: why do these two values differ?

4.2 Action history

TERMINAL

```
openstack server event list vm-web
openstack server event show vm-web <request-id>      # pick the request-id of the resize
```

Q14

What is the `request-id` used for? Why is this history useful in operations?

Before moving on to Part 5, make sure the web server responds:

TERMINAL

```
relance_web
web
```

Part 5 Instance snapshots

Log in to the VM and add a line to the home page:

TERMINAL

```
ssh -i ~/cle-lab03.pem ubuntu@$FIP
```

Inside the VM:

TERMINAL

```
echo "<p>Modifie apres le demarrage</p>" | sudo tee -a /var/www/index.html
exit
```

Back on the EC2 instance, check the change, then take a snapshot:

TERMINAL

```
web
openstack server image create vm-web --name snap-vm-web --wait
openstack image show snap-vm-web -c status -c size -c min_disk -c properties
```

Launch a new instance from the snapshot, **without user-data**, and associate a floating IP with it:

TERMINAL

```
openstack server create vm-web-clone \
    --image snap-vm-web \
    --flavor lab03.medium \
    --network net-lab03 \
    --security-group sg-lab03 \
    --key-name cle-lab03 \
    --wait

FIP_CLONE=$(openstack floating ip create public -f value -c floating_ip_address)
openstack server add floating ip vm-web-clone $FIP_CLONE
echo $FIP_CLONE
```

Wait for the boot to finish (`openstack console log show vm-web-clone | tail -5`), then test the clone:

TERMINAL

```
ping -c 3 $FIP_CLONE
curl -sS -m 5 http://$FIP_CLONE
ssh -i ~/cle-lab03.pem ubuntu@$FIP_CLONE "cat /var/www/index.html"
```

> **Q15**
> Does the clone contain the manual change? Is the web server started automatically? Explain.

Part 6 Hypervisor side: what Nova created

So far, you have handled instances through `openstack` commands, that is, through OpenStack's logical view. In this part, you will observe what actually exists on the compute host: the virtual machine defined in **libvirt** and its disk files.

All commands are run on the EC2 instance, with the `admin` credentials.

6.1 The virtual machine in libvirt

Retrieve the UUID of the instance and its libvirt name:

TERMINAL

```
ID=$(openstack server show vm-web -f value -c id)
INST=$(openstack server show vm-web -f value -c OS-EXT-SRV-ATTR:instance_name)
echo "UUID: $ID"
echo "libvirt name: $INST"
```

Display the characteristics of the VM, then the description of its disk in the XML configuration generated by Nova:

TERMINAL

```
sudo virsh dominfo $INST
sudo virsh dumpxml $INST | grep -E "<vcpu|<memory"
sudo virsh dumpxml $INST | grep -A6 "<disk"
```

6.2 The instance disk

List the instance files and examine its disk:

TERMINAL

```
sudo ls -lh /opt/stack/data/nova/instances/$ID/
sudo qemu-img info /opt/stack/data/nova/instances/$ID/disk
```

Find the `backing file` line, then examine the base image cache:

TERMINAL

```
sudo ls -lh /opt/stack/data/nova/instances/_base/
```

6.3 Comparison with the clone

Make the same observation for `vm-web-clone`:

TERMINAL

```
ID_CLONE=$(openstack server show vm-web-clone -f value -c id)
sudo qemu-img info /opt/stack/data/nova/instances/$ID_CLONE/disk
```

Compare the `backing file`, `virtual size` and `disk size` fields of the two instances.

> **Q16**
> Find the number of vCPUs and the amount of memory in the libvirt XML. Do they match the current flavor of the instance?

Cleanup

Delete the resources specific to this lab, but **keep those that will be used in Lab 04**.

a) Delete the clone, its floating IP and the snapshot

TERMINAL

```
openstack server delete vm-web-clone --wait
openstack floating ip delete $FIP_CLONE
openstack image delete snap-vm-web
```

> Deleting an instance **does not release** its floating IP: the IP stays allocated to the project until it is explicitly deleted.

b) Shelve `vm-web`

The `vm-web` instance will be reused in Lab 04 (attaching a Cinder volume). Shelve it to free the host resources between the two sessions:

TERMINAL

```
openstack server shelve vm-web
```

c) Check that no orphan resources remain

TERMINAL

```
openstack server list
openstack image list
openstack floating ip list          # no IP with a Port field set to "None"
```

If a floating IP is not associated with any port, delete it:

TERMINAL

```
openstack floating ip delete <address>
```

d) Resources kept for Lab 04

Resource	Name
Instance (shelved)	`vm-web` and its floating IP
Image	`ubuntu-24.04-minimal`
Flavors	`lab03.small`, `lab03.medium`, `lab03.private`
Network	`net-lab03`, `subnet-lab03`, `router-lab03`
Security group	`sg-lab03`
Key pair	`cle-lab03` (and the `~/cle-lab03.pem` file)
Project and user	`project-lab03`, `student-lab03`

> **End of session:** remember to stop your EC2 instance from the AWS console so as not to use up your credits. At the next start, go through Part 0 again (checking `br-ex` and the services).